

Лабораторная работа №7
Манякин Степан 6204-010302D

Задание 1

Необходимо сделать так, чтобы все объекты типа `TabulatedFunction` можно было использовать в качестве объекта-агрегата в «улучшенном цикле `for`» (вариант `for-each`), извлекаемые объекты при этом должны иметь тип `FunctionPoint`.

В интерфейсе `TabulatedFunction` добавляем необходимый родительский тип, используем при этом параметризованный тип (generic type).

В классах, реализующих интерфейс `TabulatedFunction`, добавляем требующийся метод, возвращающий объект итератора.

Классы итераторов делаем анонимными. В соответствии с паттерном «Итератор» итераторы должны работать эффективно и использовать знание о внутренней структуре объектов, а не вызывать публичные методы объекта табулированной функции.

Метод удаления всегда выбрасывает исключение `UnsupportedOperationException`.

Метод получения следующего элемента выбрасывает исключение `NoSuchElementException`, если следующего элемента нет. Возвращаемый методом объект типа `FunctionPoint` не позволяет нарушить инкапсуляцию объекта табулированной функции.

Сами изменения:

В интерфейсе:

```
import java.util.Iterator;

public interface TabulatedFunction extends Function, Cloneable,
Iterable<FunctionPoint>
```

```
Iterator<FunctionPoint> iterator()
```

В `ArrayTabulatedFunction`:

```
@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private int currentIndex = 0;

        @Override
        public boolean hasNext() {
            return currentIndex < pointsCount;
        }
    }
}
```

```

@Override
public FunctionPoint next() {
    if (!hasNext()) {
        throw new NoSuchElementException("Нет следующего элемента");
    }
    // Возвращаем КОПИЮ точки, а не оригинал
    return new FunctionPoint(points[currentIndex++]);
}

@Override
public void remove() {
    throw new UnsupportedOperationException("Удаление не поддерживается");
}
};
}

```

B LinkedListTabulatedFunction:

```

@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private FunctionNode currentNode = head.next;

        @Override
        public boolean hasNext() {
            return currentNode != head;
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException("Нет следующего элемента");
            }
            // Возвращаем КОПИЮ точки, а не оригинал
            FunctionPoint point = new FunctionPoint(currentNode.point);
            currentNode = currentNode.next;
            return point;
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException("Удаление не поддерживается");
        }
    };
}

```

Тестирование изменений в main:

```

System.out.println("=== TEST ITERATORS ===");

// Тестирование итератора
TabulatedFunction arrayFunc = new ArrayTabulatedFunction(0, 10, new
double[]{0, 1, 4, 9, 16});
System.out.println("ArrayTabulatedFunction points:");
for (FunctionPoint point : arrayFunc) {
    System.out.println(point);
}

TabulatedFunction listFunc = new LinkedListTabulatedFunction(0, 10, new

```

```
double[] {0, 1, 4, 9, 16});  
System.out.println("\nLinkedListTabulatedFunction points:");  
for (FunctionPoint point : listFunc) {  
    System.out.println(point);  
}  
  
System.out.println("\n=== ITERATOR TEST COMPLETED ===");
```

Вывод данных тестирований:

=== TEST ITERATORS ===

ArrayTabulatedFunction points:

(0.0; 0.0)

(2.5; 1.0)

(5.0; 4.0)

(7.5; 9.0)

(10.0; 16.0)

LinkedListTabulatedFunction points:

(0.0; 0.0)

(2.5; 1.0)

(5.0; 4.0)

(7.5; 9.0)

(10.0; 16.0)

=== ITERATOR TEST COMPLETED ===

Задание 2

Необходимо в пакете functions описать базовый интерфейс фабрик табулированных функций TabulatedFunctionFactory. Интерфейс объявляет три перегруженных метода TabulatedFunctioncreateTabulatedFunction(),

параметры которых соответствуют параметрам конструкторов классов табулированных функций.

```
package functions;

public interface TabulatedFunctionFactory {
    TabulatedFunction createTabulatedFunction(double leftX, double rightX,
int pointsCount);
    TabulatedFunction createTabulatedFunction(double leftX, double rightX,
double[] values);
    TabulatedFunction createTabulatedFunction(FunctionPoint[] points);
}
```

Для удобства описываем в классах ArrayTabulatedFunction и LinkedListTabulatedFunction классы фабрик ArrayTabulatedFunctionFactory и LinkedListTabulatedFunctionFactory, реализующие интерфейс фабрики и порождающие объекты соответствующих классов табулированных функций. Делаем классы фабрик вложенными и публичными.

ArrayTabulatedFunction:

```
public static class ArrayTabulatedFunctionFactory implements
TabulatedFunctionFactory {
    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double
rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double
rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points)
{
        return new ArrayTabulatedFunction(points);
    }
}
```

LinkedListTabulatedFunction:

```
public static class LinkedListTabulatedFunctionFactory implements
TabulatedFunctionFactory {
    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double
rightX, int pointsCount) {
        return new LinkedListTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double
```

```

rightX, double[] values) {
    return new LinkedListTabulatedFunction(leftX, rightX, values);
}

@Override
public TabulatedFunction createTabulatedFunction(FunctionPoint[] points)
{
    return new LinkedListTabulatedFunction(points);
}
}

```

В классе TabulatedFunctions объявляем приватное статическое поле типа TabulatedFunctionFactory и проинициализируем его объектом одного из описанных классов фабрик. Также объявляем метод setTabulatedFunctionFactory(), позволяющий заменить объект фабрики.

```

private static TabulatedFunctionFactory factory =
    new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory();

```

```

// Метод для замены фабрики
public static void setTabulatedFunctionFactory(TabulatedFunctionFactory
factory) {
    TabulatedFunctions.factory = factory;
}

```

Ещё в классе TabulatedFunctions описываем три перегруженных метода TabulatedFunction createTabulatedFunction(), возвращающих объекты табулированных функций, созданные с помощью текущей фабрики. Параметры методов соответствуют параметрам методов фабрики.

```

public static TabulatedFunction createTabulatedFunction(double leftX, double
rightX, int pointsCount) {
    return factory.createTabulatedFunction(leftX, rightX, pointsCount);
}

public static TabulatedFunction createTabulatedFunction(double leftX, double
rightX, double[] values) {
    return factory.createTabulatedFunction(leftX, rightX, values);
}

public static TabulatedFunction createTabulatedFunction(FunctionPoint[]
points) {
    return factory.createTabulatedFunction(points);
}

```

В остальных методах класса, где требуется создание объектов табулированных функций, заменяем явное создание объектов с помощью конструкторов на вызов соответствующего метода createTabulatedFunction().

Тестирование фабрик в main:

```
System.out.println("\n=== TEST FACTORY METHOD ===");

Function cos = new Cos();
TabulatedFunction tf;

// Тест 1: Фабрика по умолчанию (ArrayTabulatedFunction)
tf = TabulatedFunctions.tabulate(cos, 0, Math.PI, 11);
System.out.println("Default factory: " + tf.getClass().getSimpleName());

// Тест 2: Меняем на LinkedListTabulatedFunctionFactory
TabulatedFunctions.setTabulatedFunctionFactory(new
    LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory());
tf = TabulatedFunctions.tabulate(cos, 0, Math.PI, 11);
System.out.println("LinkedList factory: " + tf.getClass().getSimpleName());

// Тест 3: Возвращаем на ArrayTabulatedFunctionFactory
TabulatedFunctions.setTabulatedFunctionFactory(new
    ArrayTabulatedFunction.ArrayTabulatedFunctionFactory());
tf = TabulatedFunctions.tabulate(cos, 0, Math.PI, 11);
System.out.println("Array factory: " + tf.getClass().getSimpleName());

System.out.println("=== FACTORY TEST COMPLETED ===");
```

Результат теста:

=== TEST FACTORY METHOD ===

Default factory: ArrayTabulatedFunction

LinkedList factory: LinkedListTabulatedFunction

Array factory: ArrayTabulatedFunction

=== FACTORY TEST COMPLETED ===

Задание 3

Необходимо в классе TabulatedFunctions добавить ещё три перегруженных версии метода createTabulatedFunction(). Их параметры должны повторять параметры трёх аналогичных методов, основанных на использовании фабрики, но также эти методы должны получать ссылку типа Class на описание класса, объект которого требуется создать. Делаем так, чтобы в эти методы можно было передать только ссылки на классы, реализующие интерфейс TabulatedFunction.

```
//Создает табулированную функцию с помощью рефлексии
public static TabulatedFunction createTabulatedFunction(Class<?>
functionClass,
                                                    double leftX, double
rightX, int pointsCount) {
    // Проверяем, что класс реализует TabulatedFunction
```

```

        if (!TabulatedFunction.class.isAssignableFrom(functionClass)) {
            throw new IllegalArgumentException("Class must implement
TabulatedFunction");
        }

        try {
            // Находим конструктор с параметрами (double, double, int)
            Constructor<?> constructor =
functionClass.getConstructor(double.class, double.class, int.class);

            // Создаем объект
            return (TabulatedFunction) constructor.newInstance(leftX, rightX,
pointsCount);

        } catch (Exception e) {
            throw new IllegalArgumentException("Error creating tabulated
function", e);
        }
    }

    //Создает табулированную функцию с помощью рефлексии
    public static TabulatedFunction createTabulatedFunction(Class<?>
functionClass,
                                                                double leftX, double
rightX, double[] values) {
        // Проверяем, что класс реализует TabulatedFunction
        if (!TabulatedFunction.class.isAssignableFrom(functionClass)) {
            throw new IllegalArgumentException("Class must implement
TabulatedFunction");
        }

        try {
            // Находим конструктор с параметрами (double, double, double[])
            Constructor<?> constructor =
functionClass.getConstructor(double.class, double.class, double[].class);

            // Создаем объект
            return (TabulatedFunction) constructor.newInstance(leftX, rightX,
values);

        } catch (Exception e) {
            throw new IllegalArgumentException("Error creating tabulated
function", e);
        }
    }

    //Создает табулированную функцию с помощью рефлексии
    public static TabulatedFunction createTabulatedFunction(Class<?>
functionClass, FunctionPoint[] points) {
        // Проверяем, что класс реализует TabulatedFunction
        if (!TabulatedFunction.class.isAssignableFrom(functionClass)) {
            throw new IllegalArgumentException("Class must implement
TabulatedFunction");
        }

        try {
            // Находим конструктор с параметрами (FunctionPoint[])
            Constructor<?> constructor =
functionClass.getConstructor(FunctionPoint[].class);

            // Создаем объект
            return (TabulatedFunction) constructor.newInstance((Object) points);

        } catch (Exception e) {

```



```

        throw new IllegalArgumentException("Error creating tabulated
function", e);
    }
}

```

Новые методы создания объектов находят в предложенном классе конструктор с соответствующими типами параметров (например, двумя параметрами типа `double` и одним параметром типа `int` для метода, создающего объект табулированной функции по левой и правой границе области определения и количеству точек). С помощью найденного конструктора (в него должны быть переданы фактические параметры) создается объект табулированной функции. Ссылка на этот объект и возвращается из метода создания.

Если в ходе выполнения рефлексивных операций возникло исключение (не найден конструктор и т.д.), оно отловливается (используйте блок `try` с отловом нескольких типов исключений). Вместо него выбрасывается исключение `IllegalArgumentException`, причём в его конструктор передается отловленное исключение из рефлексии. Это позволит в случае возникновения ошибок определить реальную причину ошибки.

В классе `TabulatedFunctions` перегружаем методы, создающие объекты табулированных функций, добавив версии, принимающие также ссылку типа `Class` на описание класса, объект которого требуется создать. Делаем так, чтобы в эти методы можно было передать только ссылки на классы, реализующие интерфейс `TabulatedFunction`.

```

public static TabulatedFunction inputTabulatedFunction(Class<?>
functionClass, InputStream in) throws IOException {
    DataInputStream dataIn = new DataInputStream(in);

    int pointsCount = dataIn.readInt();
    if (pointsCount < 2) {
        throw new IOException("Некорректное количество точек: " +
pointsCount);
    }

    FunctionPoint[] points = new FunctionPoint[pointsCount];
    for (int i = 0; i < pointsCount; i++) {
        double x = dataIn.readDouble();
        double y = dataIn.readDouble();
        points[i] = new FunctionPoint(x, y);
    }

    // Используем рефлексию для создания объекта указанного класса
    try {
        if (!TabulatedFunction.class.isAssignableFrom(functionClass)) {
            throw new IllegalArgumentException("Class must implement
TabulatedFunction");
        }

        Constructor<?> constructor =
functionClass.getConstructor(FunctionPoint[].class);

```

```

        return (TabulatedFunction) constructor.newInstance((Object) points);
    } catch (Exception e) {
        throw new IOException("Error creating tabulated function from
stream", e);
    }
}

```

```

//Текстовый ввод табулированной функции с указанием класса через рефлексию
public static TabulatedFunction readTabulatedFunction(Class<?> functionClass,
Reader in) throws IOException {
    StreamTokenizer tokenizer = new StreamTokenizer(in);

    if (tokenizer.nextToken() != StreamTokenizer.TT_NUMBER) {
        throw new IOException("Ожидалось количество точек");
    }
    int pointsCount = (int) tokenizer.nval;

    if (pointsCount < 2) {
        throw new IOException("Некорректное количество точек: " +
pointsCount);
    }

    FunctionPoint[] points = new FunctionPoint[pointsCount];
    for (int i = 0; i < pointsCount; i++) {
        if (tokenizer.nextToken() != StreamTokenizer.TT_NUMBER) {
            throw new IOException("Ожидалась координата X точки " + (i + 1));
        }
        double x = tokenizer.nval;

        if (tokenizer.nextToken() != StreamTokenizer.TT_NUMBER) {
            throw new IOException("Ожидалась координата Y точки " + (i + 1));
        }
        double y = tokenizer.nval;

        points[i] = new FunctionPoint(x, y);
    }

    try {
        if (!TabulatedFunction.class.isAssignableFrom(functionClass)) {
            throw new IllegalArgumentException("Class must implement
TabulatedFunction");
        }

        Constructor<?> constructor =
functionClass.getConstructor(FunctionPoint[].class);
        return (TabulatedFunction) constructor.newInstance((Object) points);

    } catch (Exception e) {
        throw new IOException("Error creating tabulated function from
reader", e);
    }
}

```

Проверка в main:

```

System.out.println("\n=== TEST REFLECTION METHODS ===");

    TabulatedFunction f;

    // Тест 1: Создание ArrayTabulatedFunction через рефлексию
    f =
TabulatedFunctions.createTabulatedFunction(ArrayTabulatedFunction.class, 0,
10, 3);
    System.out.println("Array via reflection: " +
f.getClass().getSimpleName());
    System.out.println(f);

    // Тест 2: Создание LinkedListTabulatedFunction через рефлексию
    f =
TabulatedFunctions.createTabulatedFunction(LinkedListTabulatedFunction.class,
0, 10, new double[]{0, 5, 10});
    System.out.println("LinkedList via reflection: " +
f.getClass().getSimpleName());
    System.out.println(f);

    // Тест 3: Табулирование с рефлексией
    f = TabulatedFunctions.tabulate(LinkedListTabulatedFunction.class, new
Sin(), 0, Math.PI, 11);
    System.out.println("Tabulate with reflection: " +
f.getClass().getSimpleName());
    System.out.println(f);

    System.out.println("=== REFLECTION TEST COMPLETED ===");

    // =====
    System.out.println("\n=== TEST BINARY INPUT/OUTPUT WITH FACTORY ===");

    try {
        TabulatedFunction originalFunc = new ArrayTabulatedFunction(
0, 10, new double[]{0, 1, 4, 9, 16, 25});

        // Записываем в байтовый поток
        ByteArrayOutputStream byteOut = new ByteArrayOutputStream();
        TabulatedFunctions.outputTabulatedFunction(originalFunc, byteOut);

        // Читаем обратно с фабрикой по умолчанию (Array)
        ByteArrayInputStream byteIn = new
ByteArrayInputStream(byteOut.toByteArray());
        TabulatedFunction funcFromBinary =
TabulatedFunctions.inputTabulatedFunction(byteIn);
        System.out.println("From binary stream (factory): " +
funcFromBinary.getClass().getSimpleName());

        // Читаем с указанием конкретного класса через рефлексию
        byteIn = new ByteArrayInputStream(byteOut.toByteArray());
        TabulatedFunction funcFromBinaryReflection =
TabulatedFunctions.inputTabulatedFunction(
LinkedListTabulatedFunction.class, byteIn);
        System.out.println("From binary stream (reflection): " +
funcFromBinaryReflection.getClass().getSimpleName());

        System.out.println("Original: " + originalFunc);
        System.out.println("From binary (factory): " + funcFromBinary);
        System.out.println("From binary (reflection): " +
funcFromBinaryReflection);

    } catch (Exception e) {
        System.err.println("Binary I/O error: " + e.getMessage());
        e.printStackTrace();
    }

```

```

    }

    System.out.println("=== BINARY I/O TEST COMPLETED ===");

    // =====
    System.out.println("\n=== TEST TEXT INPUT/OUTPUT ===");

    try {
        TabulatedFunction originalFunc = new ArrayTabulatedFunction(
            0, 10, new double[]{0, 1, 4, 9, 16});

        // Записываем в символьный поток
        StringWriter stringWriter = new StringWriter();
        TabulatedFunctions.writeTabulatedFunction(originalFunc,
stringWriter);
        String serialized = stringWriter.toString();

        // Читаем обратно с фабрикой по умолчанию
        StringReader stringReader = new StringReader(serialized);
        TabulatedFunction funcFromText =
TabulatedFunctions.readTabulatedFunction(stringReader);
        System.out.println("From text stream (factory): " +
funcFromText.getClass().getSimpleName());

        // Читаем с указанием конкретного класса через рефлексию
        stringReader = new StringReader(serialized);
        TabulatedFunction funcFromTextReflection =
TabulatedFunctions.readTabulatedFunction(
            LinkedListTabulatedFunction.class, stringReader);
        System.out.println("From text stream (reflection): " +
funcFromTextReflection.getClass().getSimpleName());

        System.out.println("Original: " + originalFunc);
        System.out.println("From text (factory): " + funcFromText);
        System.out.println("From text (reflection): " +
funcFromTextReflection);

    } catch (Exception e) {
        System.err.println("Text I/O error: " + e.getMessage());
        e.printStackTrace();
    }

    System.out.println("=== TEXT I/O TEST COMPLETED ===");
}

```

Вывод main с данной проверкой:

=== TEST REFLECTION METHODS ===

Array via reflection: ArrayTabulatedFunction

{(0.0; 0.0), (5.0; 0.0), (10.0; 0.0)}

LinkedList via reflection: LinkedListTabulatedFunction

{(0.0; 0.0), (5.0; 5.0), (10.0; 10.0)}

Tabulate with reflection: LinkedListTabulatedFunction

{(0.0; 0.0), (0.3141592653589793; 0.3090169943749474), (0.6283185307179586; 0.5877852522924731), (0.9424777960769379; 0.8090169943749475), (1.2566370614359172; 0.9510565162951535), (1.5707963267948966; 1.0), (1.8849555921538759; 0.9510565162951536), (2.199114857512855; 0.8090169943749475), (2.5132741228718345; 0.5877852522924732), (2.827433388230814; 0.3090169943749475), (3.141592653589793; 1.2246467991473532E-16)}

=== REFLECTION TEST COMPLETED ===

=== TEST BINARY INPUT/OUTPUT WITH FACTORY ===

From binary stream (factory): ArrayTabulatedFunction

From binary stream (reflection): LinkedListTabulatedFunction

Original: {(0.0; 0.0), (2.0; 1.0), (4.0; 4.0), (6.0; 9.0), (8.0; 16.0), (10.0; 25.0)}

From binary (factory): {(0.0; 0.0), (2.0; 1.0), (4.0; 4.0), (6.0; 9.0), (8.0; 16.0), (10.0; 25.0)}

From binary (reflection): {(0.0; 0.0), (2.0; 1.0), (4.0; 4.0), (6.0; 9.0), (8.0; 16.0), (10.0; 25.0)}

=== BINARY I/O TEST COMPLETED ===

=== TEST TEXT INPUT/OUTPUT ===

From text stream (factory): ArrayTabulatedFunction

From text stream (reflection): LinkedListTabulatedFunction

Original: {(0.0; 0.0), (2.5; 1.0), (5.0; 4.0), (7.5; 9.0), (10.0; 16.0)}

From text (factory): {(0.0; 0.0), (2.5; 1.0), (5.0; 4.0), (7.5; 9.0), (10.0; 16.0)}

From text (reflection): {(0.0; 0.0), (2.5; 1.0), (5.0; 4.0), (7.5; 9.0), (10.0; 16.0)}

=== TEXT I/O TEST COMPLETED ===

Ниже представлен полный вывод main:

=== TEST ITERATORS ===

ArrayTabulatedFunction points:

(0.0; 0.0)

(2.5; 1.0)

(5.0; 4.0)

(7.5; 9.0)

(10.0; 16.0)

LinkedListTabulatedFunction points:

(0.0; 0.0)

(2.5; 1.0)

(5.0; 4.0)

(7.5; 9.0)

(10.0; 16.0)

=== ITERATOR TEST COMPLETED ===

=== TEST FACTORY METHOD ===

Default factory: ArrayTabulatedFunction

LinkedList factory: LinkedListTabulatedFunction

Array factory: ArrayTabulatedFunction

=== FACTORY TEST COMPLETED ===

=== TEST REFLECTION METHODS ===

Array via reflection: ArrayTabulatedFunction

{{(0.0; 0.0), (5.0; 0.0), (10.0; 0.0)}}

LinkedList via reflection: LinkedListTabulatedFunction

{{(0.0; 0.0), (5.0; 5.0), (10.0; 10.0)}}

Tabulate with reflection: LinkedListTabulatedFunction

{{(0.0; 0.0), (0.3141592653589793; 0.3090169943749474), (0.6283185307179586;
0.5877852522924731), (0.9424777960769379; 0.8090169943749475),
(1.2566370614359172; 0.9510565162951535), (1.5707963267948966; 1.0),
(1.8849555921538759; 0.9510565162951536), (2.199114857512855;
0.8090169943749475), (2.5132741228718345; 0.5877852522924732),
(2.827433388230814; 0.3090169943749475), (3.141592653589793;
1.2246467991473532E-16)}}

=== REFLECTION TEST COMPLETED ===

=== TEST BINARY INPUT/OUTPUT WITH FACTORY ===

From binary stream (factory): ArrayTabulatedFunction

From binary stream (reflection): LinkedListTabulatedFunction

Original: {{(0.0; 0.0), (2.0; 1.0), (4.0; 4.0), (6.0; 9.0), (8.0; 16.0), (10.0; 25.0)}}

From binary (factory): {{(0.0; 0.0), (2.0; 1.0), (4.0; 4.0), (6.0; 9.0), (8.0; 16.0),
(10.0; 25.0)}}

From binary (reflection): {{(0.0; 0.0), (2.0; 1.0), (4.0; 4.0), (6.0; 9.0), (8.0; 16.0),
(10.0; 25.0)}}

=== BINARY I/O TEST COMPLETED ===

=== TEST TEXT INPUT/OUTPUT ===

From text stream (factory): ArrayTabulatedFunction

From text stream (reflection): LinkedListTabulatedFunction

Original: {(0.0; 0.0), (2.5; 1.0), (5.0; 4.0), (7.5; 9.0), (10.0; 16.0)}

From text (factory): {(0.0; 0.0), (2.5; 1.0), (5.0; 4.0), (7.5; 9.0), (10.0; 16.0)}

From text (reflection): {(0.0; 0.0), (2.5; 1.0), (5.0; 4.0), (7.5; 9.0), (10.0; 16.0)}

=== TEXT I/O TEST COMPLETED ===

Process finished with exit code 0

Спасибо за внимание!